W11. General-Purpose computing on Graphics Processing Units (GPGPU)

- Part. 2-

Department of Smart Mobility Engineering

Inwook, Shim

인하대학교 INHA UNIVERSITY

Performance Result!

- 왜 안빠름?
- DATA_SIZE 늘리며 성능 비교 해보기

연산틀림

```
[70] The resutls is not matched! (9, 0)
[71] The resutls is not matched! (11, 0)
[72] The resutls is not matched! (12, 0)
[73] The resutls is not matched! (8, 0)
[74] The resutls is not matched! (13, 0)
[75] The resutls is not matched! (9, 0)
[76] The resutls is not matched! (9, 0)
[77] The resutls is not matched! (13, 0)
[78] The resutls is not matched! (7, 0)
[79] The resutls is not matched! (6, 0)
[80] The resutls is not matched! (17, 0)
[81] The resutls is not matched! (12, 0)
[82] The resutls is not matched! (1, 0)
[83] The resutls is not matched! (3, 0)
[84] The resutls is not matched! (8, 0)
[85] The resutls is not matched! (8, 0)
[86] The resutls is not matched! (13, 0)
[87] The resutls is not matched! (4, 0)
[88] The resutls is not matched! (12, 0)
[89] The resutls is not matched! (15, 0)
[90] The resutls is not matched! (3, 0)
[91] The resutls is not matched! (15, 0)
[92] The resutls is not matched! (12, 0)
[93] The resutls is not matched! (5, 0)
[94] The resutls is not matched! (13, 0)
```

코드 셀 출력 작업

```
VectorSum on Host: 0.001521 ms

Data Trans. : Host -> Device: 0.033843 ms
Computation: 37.4369 ms
Data Trans. : Device -> Host: 0.026216 ms
Cuda Total: 37.5472 ms
```

CUDA Thread

CUDA Threads

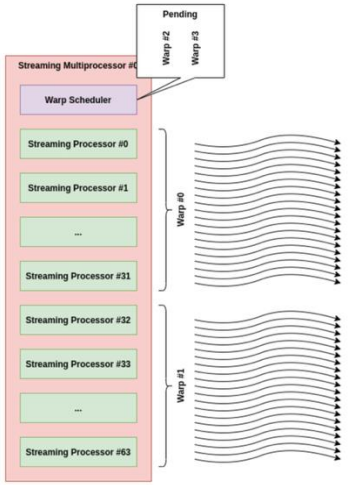
KERNEL.

```
__global__ void vecAdd(int *_a, int *_b, int *_c) {
        //printf("BlockID: %d, thread ID: %d\n", blockIdx.x, threadIdx.x);
        int tID = threadIdx.x;
        _c[tID] = _a[tID] + _b[tID];
}
```

All threads share this code.

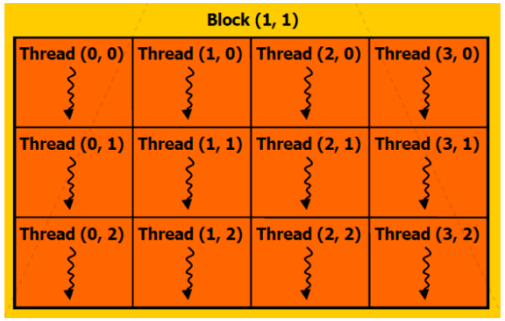
KERNEL CALL.

```
vecAdd<<<1, NUM_DATA >>>(d_a, d_b, d_c);
```



- **Thread**: basic processing unit
- **Warp**: 32 Threads, basic execution unit, controlled by the same instruction
- **Block**: Group of threads (3D까지 컨트롤 가능)

  Threads in a block have different id. (`threadIdx`)
- **Grid**: Group of blocks (3D까지 컨트롤 가능), `blockIdx`



CUDA Threads

KERNEL.

```
__global__ void vecAdd(int *_a, int *_b, int *_c) {
        //printf("BlockID: %d, thread ID: %d\n", blockIdx.x, threadIdx.x);
        int tID = threadIdx.x;
        _c[tID] = _a[tID] + _b[tID];
}
```

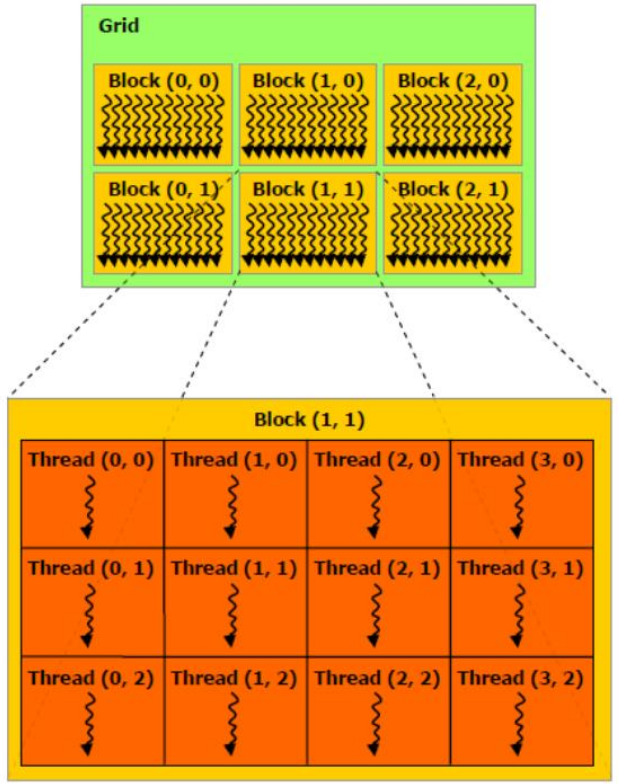
All threads share this code.

KERNEL CALL.

```
vecAdd<<<1, NUM_DATA >>>(d_a, d_b, d_c);
```

- **Thread**: basic processing unit
- **Warp**: 32 Threads, basic execution unit, controlled by the same ins
- **Block**: Group of threads (3D까지 컨트롤 가능)

  Threads in a block have different id. (`threadIdx`)
- **Grid**: Group of blocks (3D까지 컨트롤 가능), `blockIdx`

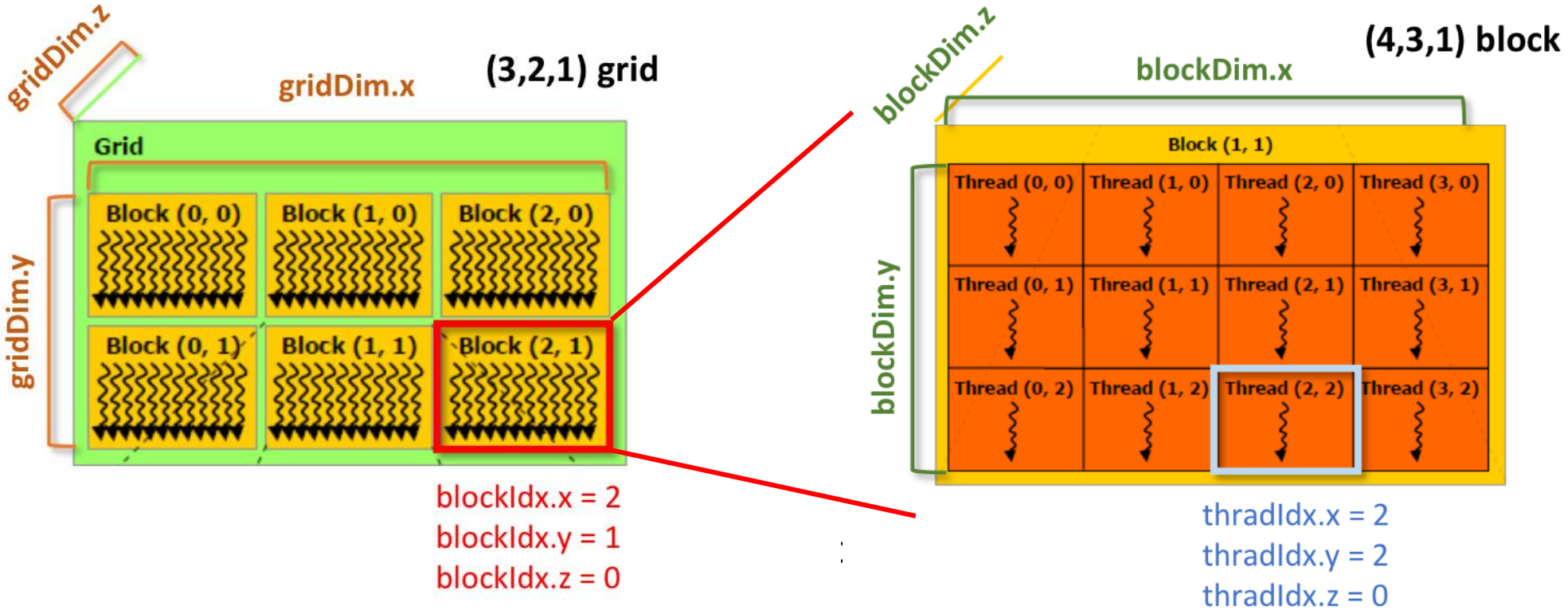


[From CUDA C Programming Guide]

CUDA Threads

- **Block**
  - 사용자가 직접 크기를 정함 (e.g., 128, 256, 512 threads)
  - `__syncthreads()` 로 block 내부 thread 간 동기화 가능
  - shared memory 를 block 단위로 공유
- **Warp**: 하드웨어 수준에서 block을 잘라서 만든 실행 단위
  - 동일한 명령을 동시에 실행하는 threads 묶음
  - NVIDIA GPU 기준: 1 warp == 32 Threads (사용자가 변경할 수 없음.)
  - block dim = (256,1,1) 이면, 256/8 = 32 warps
  - 분기 (If) 에 따라서 threads 간 다른 역할을 수행하면 warp divergence >> 성능저하
- `block` 크기를 정하면, 그 안에서 자동으로 `warps` 가 생김 (논리적 그룹)
- `warp` 실제 실행 단위 (스케쥴러가 `warp`를 번갈아 가면서 실행)

CUDA Threads



- **`gridDim`**

  - grid 안의 block 수 결정

- **`blockIdx`**

  - grid 안에서의 각 block 의 id.

- **`blockDim`**

  - block 안에서 thread 수 결정

- **`threadIdx`**

  - block 안에서의 각 thread 의 id.

CUDA Threads

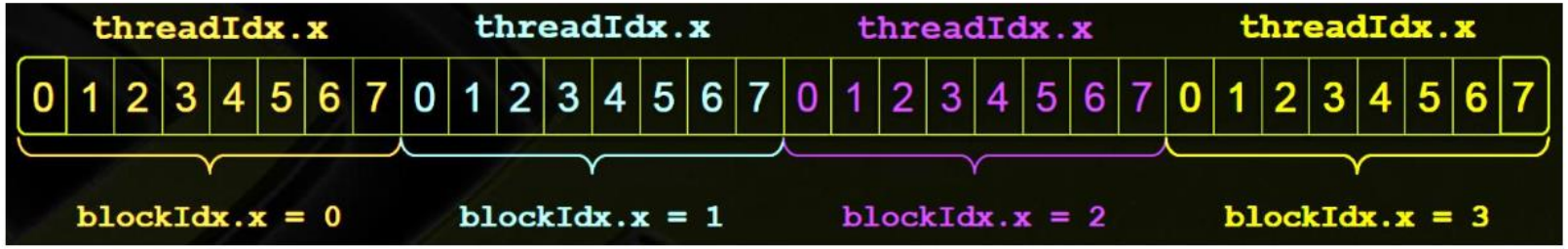
	Compute Capability										
Technical Specifications	**3.0**	**3.2**	**3.5**	**3.7**	**5.0**	**5.2**	**5.3**	**6.0**	**6.1**	**6.2**	**7.0**
Maximum number of resident grids per device (Concurrent Kernel Execution)	16	4	32				16	128	32	16	128
Maximum dimensionality of grid of thread blocks	3										
Maximum x-dimension of a grid of thread blocks	$2^{31}-1$										
Maximum y- or z-dimension of a grid of thread blocks	65535										
Maximum dimensionality of thread block	3										
Maximum x- or y-dimension of a block	1024										
Maximum z-dimension of a block	64										
Maximum number of threads per block	1024										
Warp size	32										

[From CUDA C Programming Guide]

CUDA Threads

- Set the thread layout : set the dimensions of grid and block.

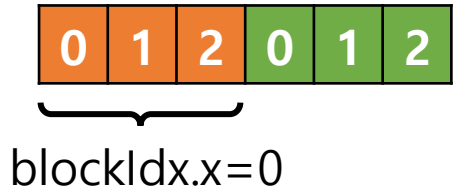
```
dim3 dimGrid(4, 1, 1);
dim3 dimBlock(8, 1, 1);
vecAdd <<<dimGrid, dimBlock>>>(d_a, d_b, d_c);
```



CUDA Threads

```
# @title CheckDim.cu
%%cuda
#include <stdio.h>

__global__ void checkIndex(void) {
    printf("threadIdx:(%d, %d, %d) blockIdx:(%d, %d, %d) blockDim:(%d, %d, %d) gridDim:(%d, %d, %d)\n"
          , threadIdx.x, threadIdx.y, threadIdx.z
          , blockIdx.x, blockIdx.y, blockIdx.z
          , blockDim.x, blockDim.y, blockDim.z
          , gridDim.x, gridDim.y, gridDim.z);
}
int main(int argc, char **argv)
{
    int nElem = 6;

    dim3 block(3);
    dim3 grid((nElem + block.x - 1) / block.x);

    printf("grid.x %d grid.y %d grid.z %d\n", grid.x, grid.y, grid.z);
    printf("block.x %d block.y %d block.z %d\n", block.x, block.y, block.z);

    checkIndex <<<grid, block>>> ();
    cudaDeviceReset();

return(0);
}
```

사용하던 GPU 상태를 컨텍스트를 종료하고 초기 상태로 리셋함.
- GPU 메모리 해제: cudamalloc 으로 할당한 메모리, 텍스쳐 등
- 보통 프로그램 끝날 때, 메모리 누수없이 종료.
- 프로파일러 (nvprof, Nsight 등) 사용시



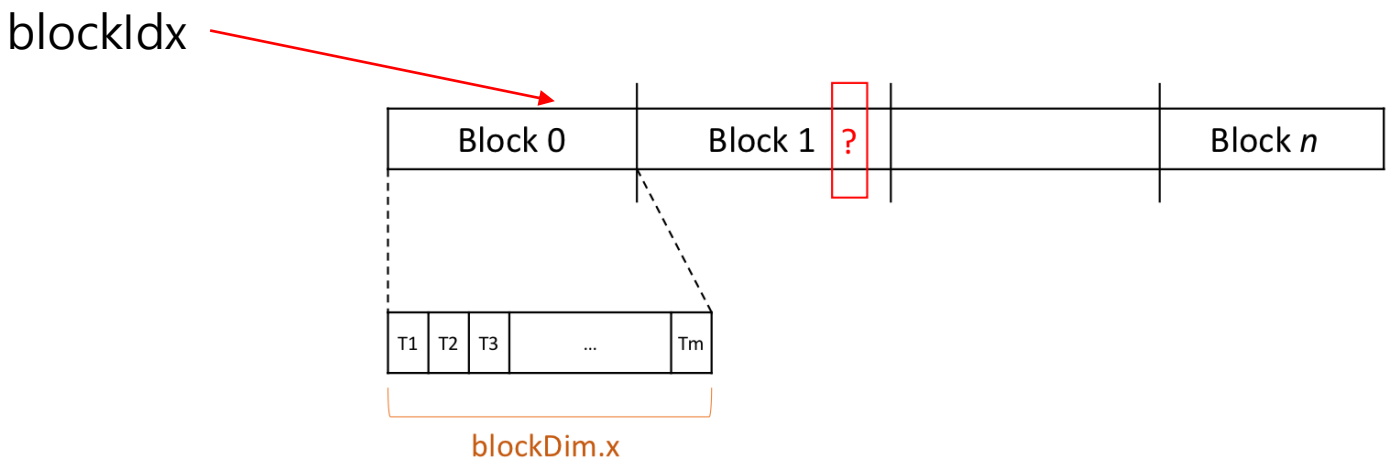
CUDA Threads : Rethink vector sum kernel.

KERNEL.

```
__global__ void vecAdd(int *_a, int *_b, int *_c) {
        //printf("BlockID: %d, thread ID: %d\n", blockIdx.x, threadIdx.x);
        int tID = threadIdx.x;
        _c[tID] = _a[tID] + _b[tID];
}
```

KERNEL CALL.

```
vecAdd<<<1, NUM_DATA >>>(d_a, d_b, d_c);
```



CUDA Threads : Rethink vector sum kernel.

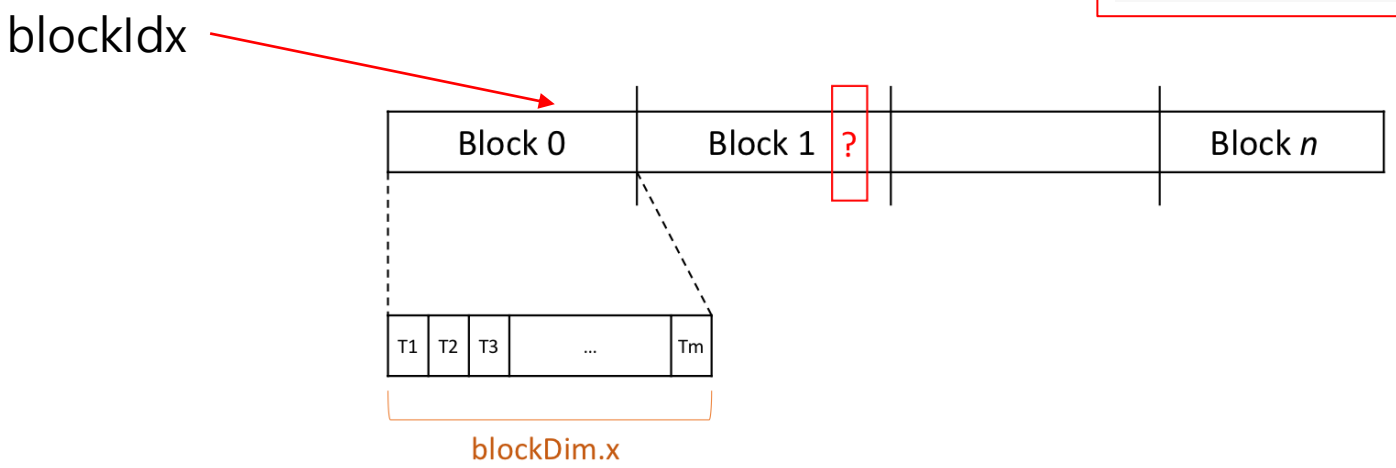
KERNEL.

```
__global__ void vecAdd(int *_a, int *_b, int *_c) {
        //printf("BlockID: %d, thread ID: %d\n", blockIdx.x, threadIdx.x);
        int tID = blockIdx.x * blockDim.x + threadIdx.x;
        _c[tID] = _a[tID] + _b[tID];
}
```

KERNEL CALL.

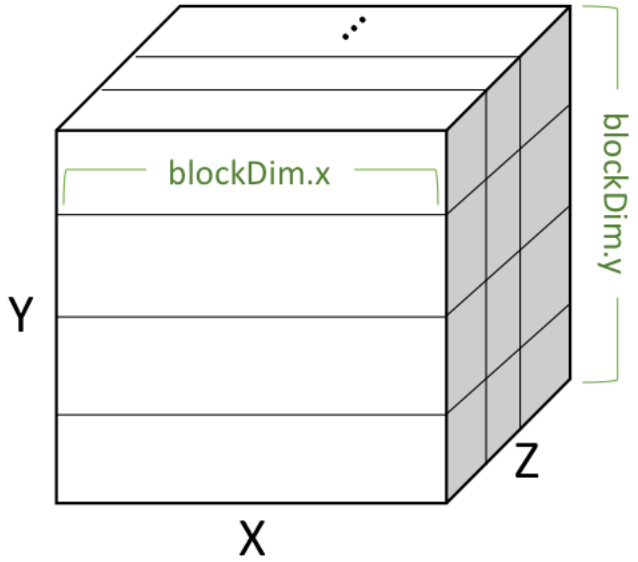
```
vecAdd<<<1, NUM_DATA >>>(d_a, d_b, d_c);
```

```
#define NUM_THREAD 1024
#define NUM_DATA 20480
…
dim3 dimGrid(NUM_DATA / NUM_THREAD, 1, 1);
dim3 dimBlock(NUM_THREAD, 1, 1);
// <<< BlockNum, ThreadNum >>>
// 1개의 블록은 N개의 쓰레드로 구성
vecAdd<<<dimGrid, dimBlock>>>(d_a, d_b, d_c);
```



CUDA Threads : Global thread ID in 1D Grid

- Block 내의 thread id (=TID_BLOCK)
  - In 1D block
    - TID_BLOCK = threadIdx.x
  - In 2D block (=TID_2D_BLOCK)
    - TID_BLOCK = (blockDim.x * threadIdx.y) + threadIdx.x
  - In 3D block
    - TID_BLOCK = ((blockDim.y*blockDim.x)*threadIdx.z) + TID_2D_BLOCK



- Grid 내의 thread id
  - (blockIdx.x * (blockDim.x * blockDim.y * blockDim.z)) + TID_BLOCK

Block 내의 최대 thread 수 = MAX_THREAD_IN_BLOCK

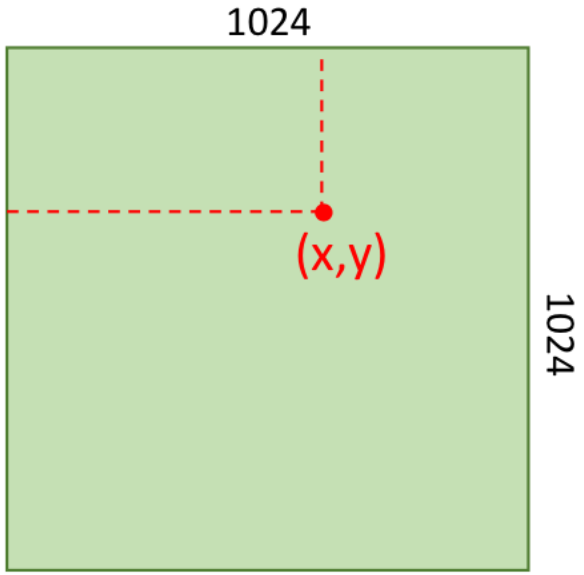
CUDA Threads : Global thread ID in 1D Grid

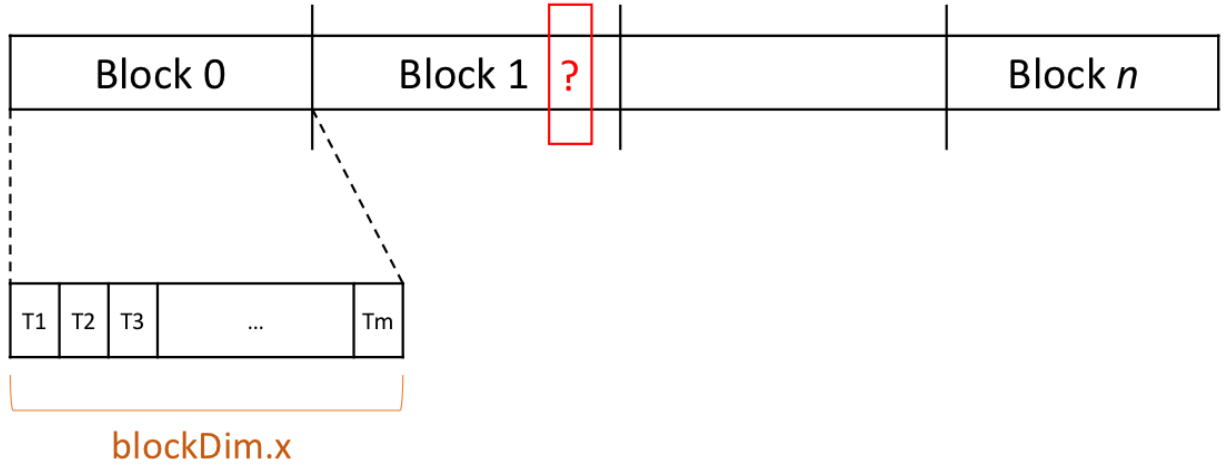
- Grid 내의 thread id (TID_GRID)
  - in 1D grid (= TID_1D_GRID)
    - (blockIdx.x * (blockDim.x * blockDim.y * blockDim.z)) + TID_BLOCK

Block 내의 최대 thread 수 = MAX_THREAD_IN_BLOCK

  - in 2D grid (TID_2D_GRID)
    - blockIdx.y * (gridDim.x * MAX_THREAD_IN_BLOCK)) + TID_1D_GRID
  - in 3D grid
    - blockIdx.z * (gridDim.y * gridDim.x * MAX_THREAD_IN_BLOCK)) + TID_2D_GRID

CUDA Threads : example) Thread indexing for 2D matrix





The matrix (tensor) is stored in the device memory as a form of 1D array.

```
i = blockIdx.x * blockDim.x + threadIdx.x;
j = blockIdx.y * blockDim.y + threadIdx.y;
Index = j * blockDim.x + i
```

Tip.
계산할 필요없는 thread 는 skip

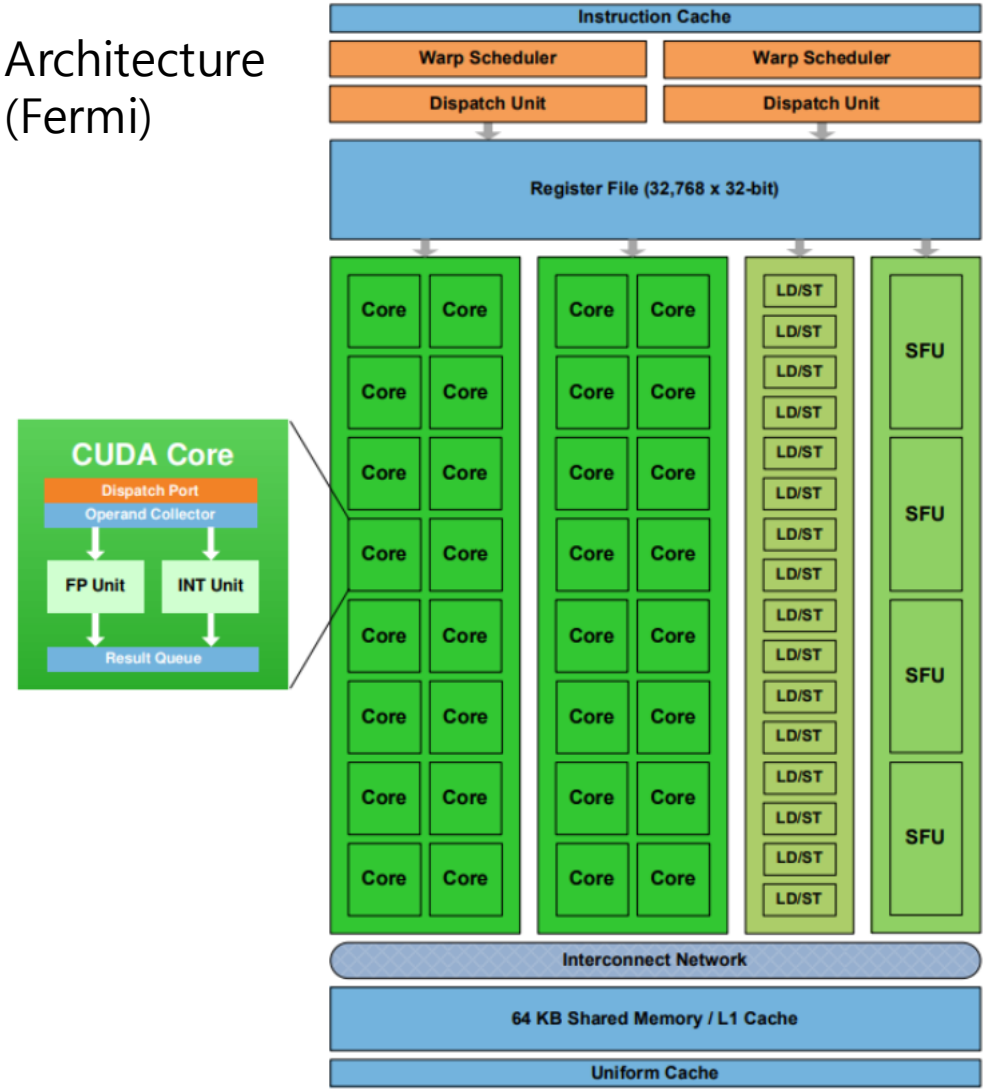
```
__global__ void vecAdd(int numThreads, int …)
{
    int tID = blockIdx.x*blockDim.x + threadIdx.x;
    if (tID > numThreads)
        return;
    ...
}
```

CUDA HW Specification

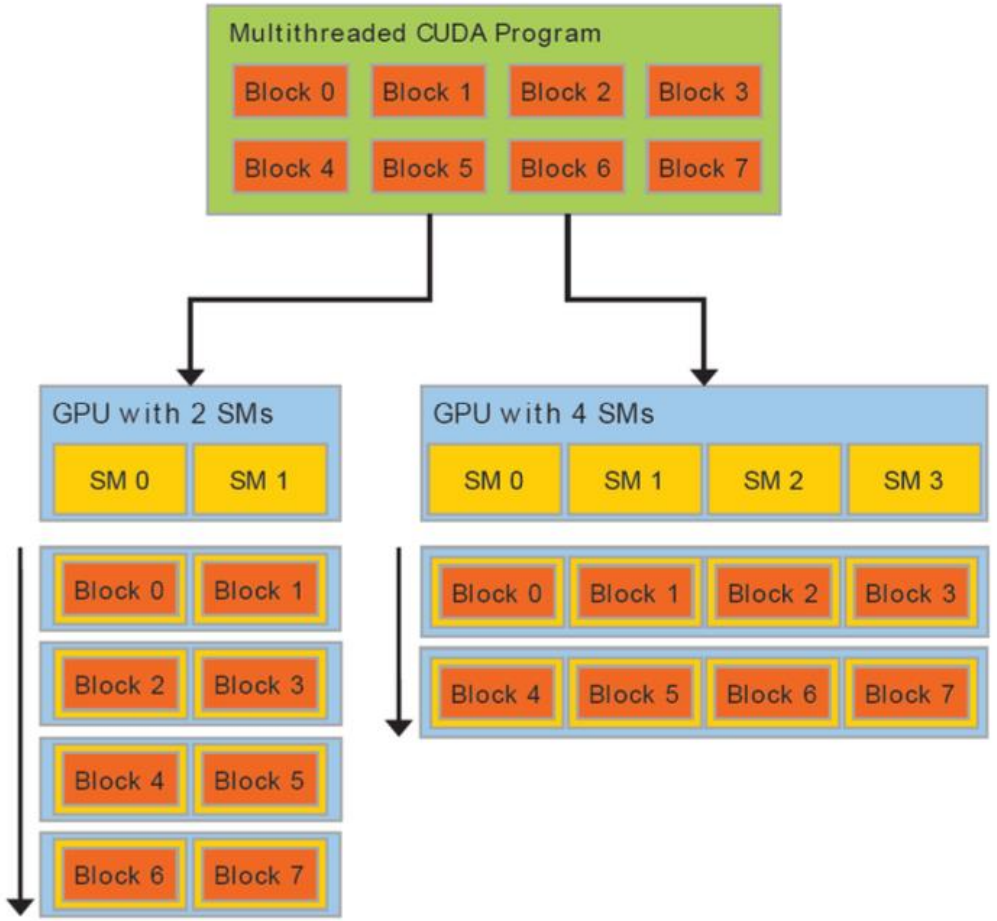
- CUDA core
  - Basic Processing Unit
  - 하나의 thread 를 처리함
  - Register, Shared Memory

- Streaming Multiprocessor (SM)
  - 32개의 CUDA core 로 구성 (우측 그림 참고)
  - 하나의 thread **block** 을 처리

GPU Architecture
(Fermi)



CUDA HW Specification

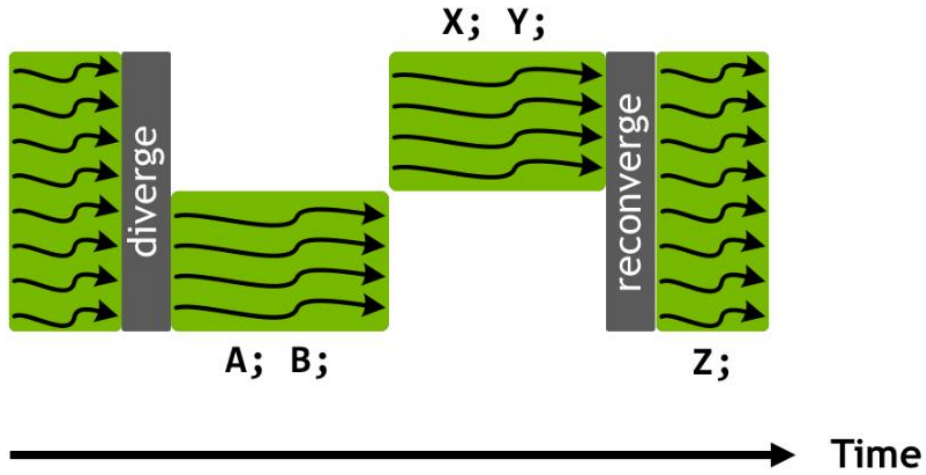


- Grid/Kernel
  - Kernel launch 시 생성
  - GPU를 사용하는 단위
- Block
  - 각 Streaming Multiprocessor 에 block 들이 할당
  - Active block : 현재 SM에서 처리중인 block
  - Block 당 자원 사용량에 의해 결정.
- Warp
  - 각 block 은 warp 단위로 분할
  - warp 단위의 execution context (program counter, register 등), SM 내의 register 를 활용
  - warp divergence: 한 warp(32개의 thread) 들이 분기(branch e.g., if) 연산이 있으면, 분기별로 serial 하게 수행
  - Zero context switching overhead를 위해 thread 갯수를 적절히 조절

CUDA HW Specification

- Warp divergence

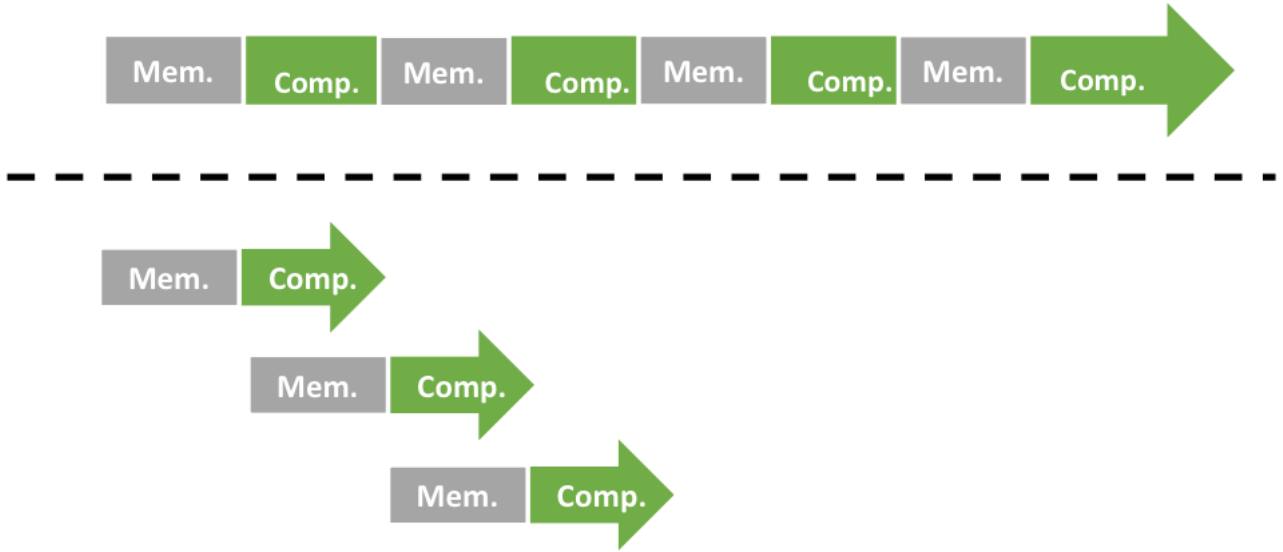
```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```



- Warp 안의 thread들이 서로 다른 분기를 따르는 경우, 분기별로 직렬화 되어 처리됨
- GPU 알고리즘 성능에 매우 안좋음! → 분기를 최대한 피해야 함

Massive Parallelism for Latency Hiding

- GPU를 녹여라!



CUDA HW Specification

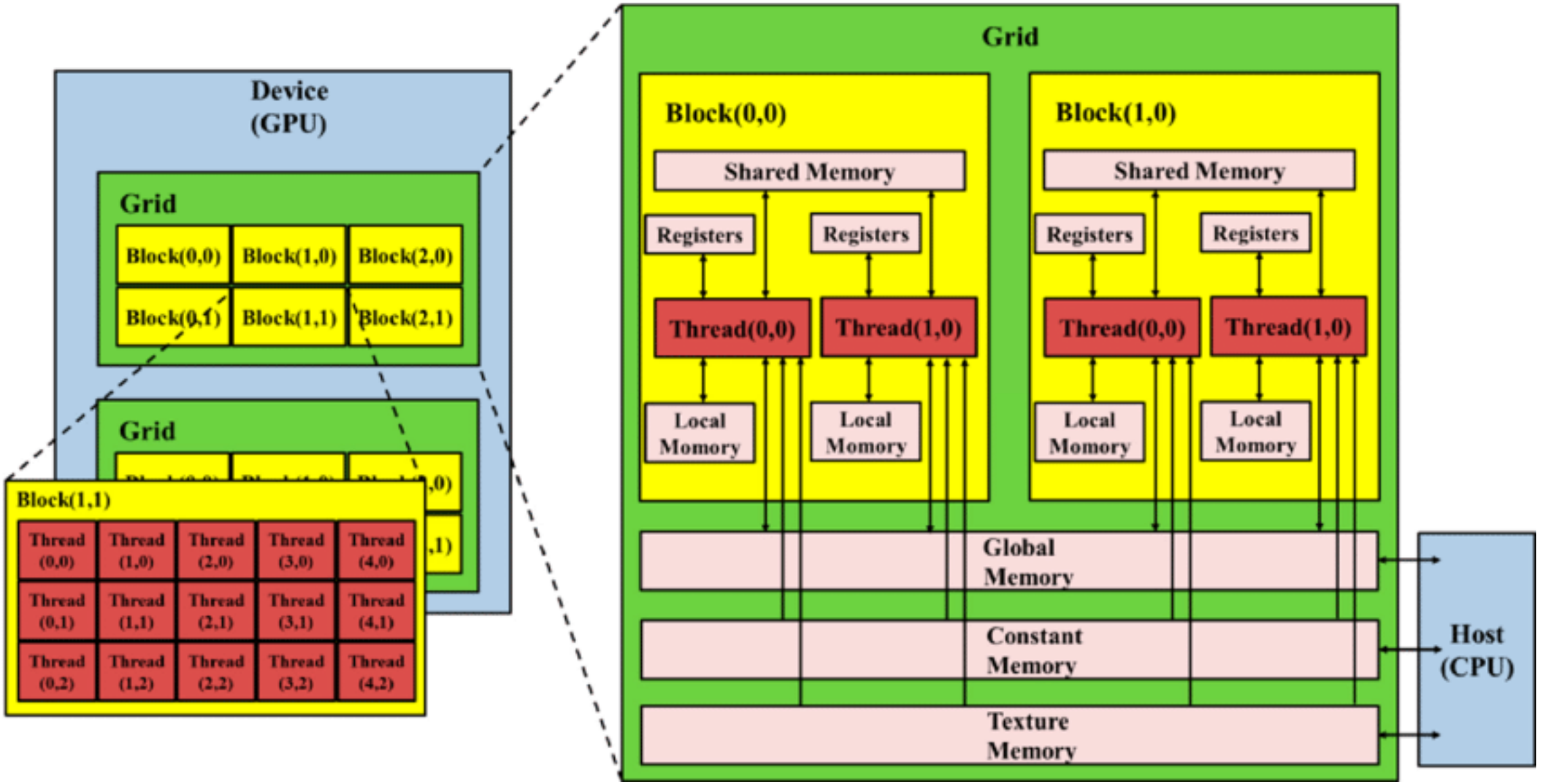
	RTX 2080 Ti Founder's Edition	RTX 2080 Founder's Edition	GTX 1080 Ti
CUDA Cores	4352	2944	3584
Gigarays	10 Gigarays/s	8 Gigarays/s	N/A
Base Clock (MHz)	1350	1515	1481
Boost Clock (MHz)	1635 (OC)	1800 (OC)	1582
Total Video Memory	11GB GDDR6	8GB GDDR6	11GB GDDR5X
Memory Interface	352-bit	256-bit	352-bit
Memory Speed	14Gb/s	14Gb/s	11Gb/s
Total Memory Bandwidth	616GB/s	448GB/s	484GB/s
Power Connector	2x Eight-pin	Eight-pin, six-pin	Eight-pin, six-pin
Thermal Design Power (TDP)	260w	225w	250w

CUDA HW Specification

```
./deviceQuery Starting...

 CUDA Device Query (Runtime API) version (CUDART static linking)

Detected 1 CUDA Capable device(s)

Device 0: "Tesla T4"
  CUDA Driver Version / Runtime Version          10.0 / 10.0
  CUDA Capability Major/Minor version number:    7.5
  Total amount of global memory:                 15080 MBytes (15812263936 bytes)
  (40) Multiprocessors, ( 64) CUDA Cores/MP:     2560 CUDA Cores
  GPU Max Clock rate:                            1590 MHz (1.59 GHz)
  Memory Clock rate:                             5001 Mhz
  Memory Bus Width:                              256-bit
  L2 Cache Size:                                 4194304 bytes
  Maximum Texture Dimension Size (x,y,z)         1D=(131072), 2D=(131072, 65536), 3D=(16384, 16384, 16384)
  Maximum Layered 1D Texture Size, (num) layers  1D=(32768), 2048 layers
  Maximum Layered 2D Texture Size, (num) layers  2D=(32768, 32768), 2048 layers
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:       49152 bytes
  Total number of registers available per block: 65536
  Warp size:                                     32
  Maximum number of threads per multiprocessor:  1024
  Maximum number of threads per block:           1024
  Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
  Max dimension size of a grid size    (x,y,z): (2147483647, 65535, 65535)
  Maximum memory pitch:                          2147483647 bytes
  Texture alignment:                             512 bytes
  Concurrent copy and kernel execution:          Yes with 3 copy engine(s)
  Run time limit on kernels:                     No
  Integrated GPU sharing Host Memory:            No
  Support host page-locked memory mapping:       Yes
  Alignment requirement for Surfaces:            Yes
  Device has ECC support:                        Enabled
  Device supports Unified Addressing (UVA):      Yes
  Device supports Compute Preemption:            Yes
  Supports Cooperative Kernel Launch:            Yes
  Supports MultiDevice Co-op Kernel Launch:      Yes
  Device PCI Domain ID / Bus ID / location ID:   0 / 0 / 4
  Compute Mode:
     < Default (multiple host threads can use ::cudaSetDevice() with device simultaneously) >

deviceQuery, CUDA Driver = CUDART, CUDA Driver Version = 10.0, CUDA Runtime Version = 10.0, NumDevs = 1
Result = PASS
```

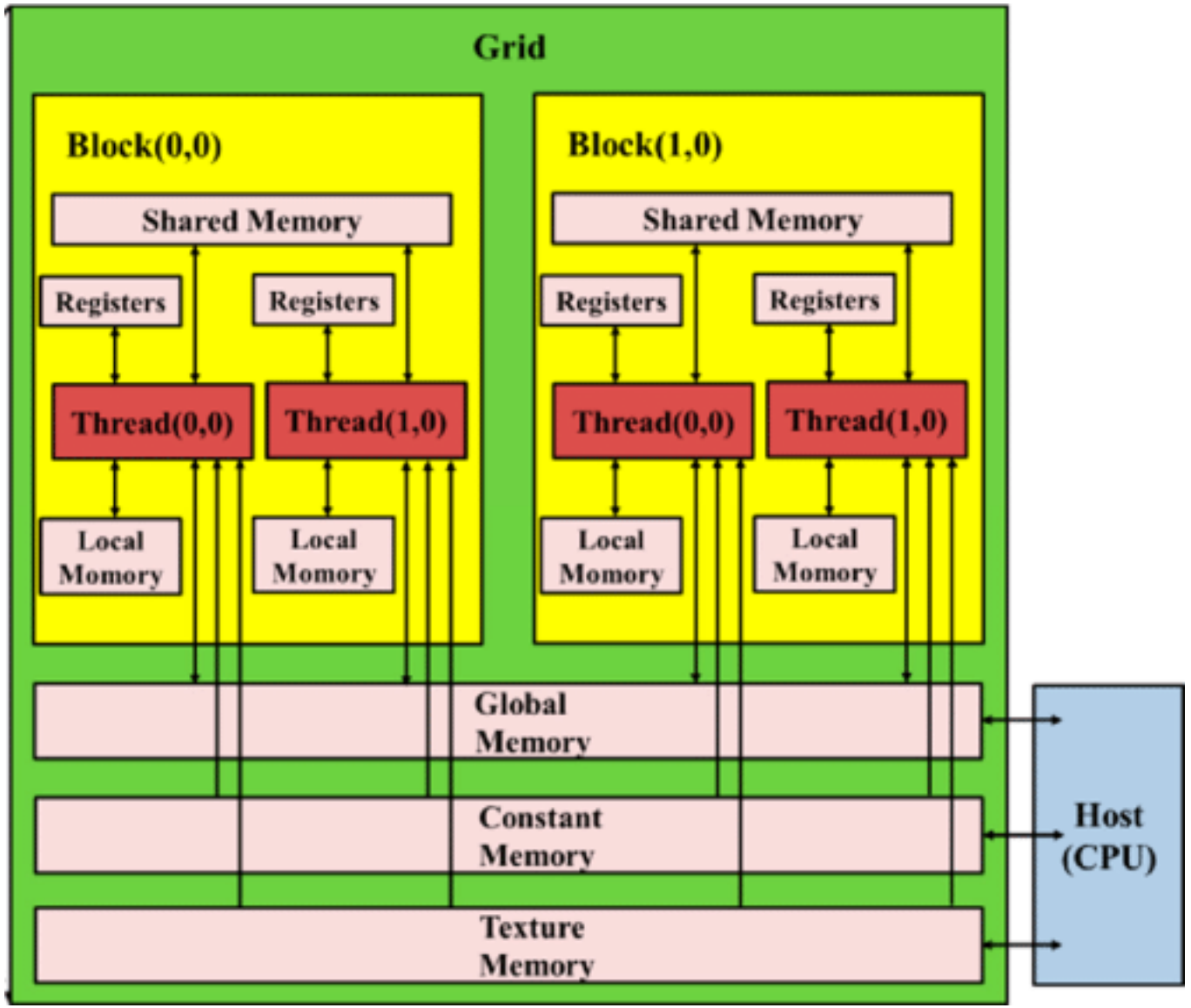
CUDA Memory Hierarchy

CUDA Memory Hierarchy



CUDA Memory Hierarchy

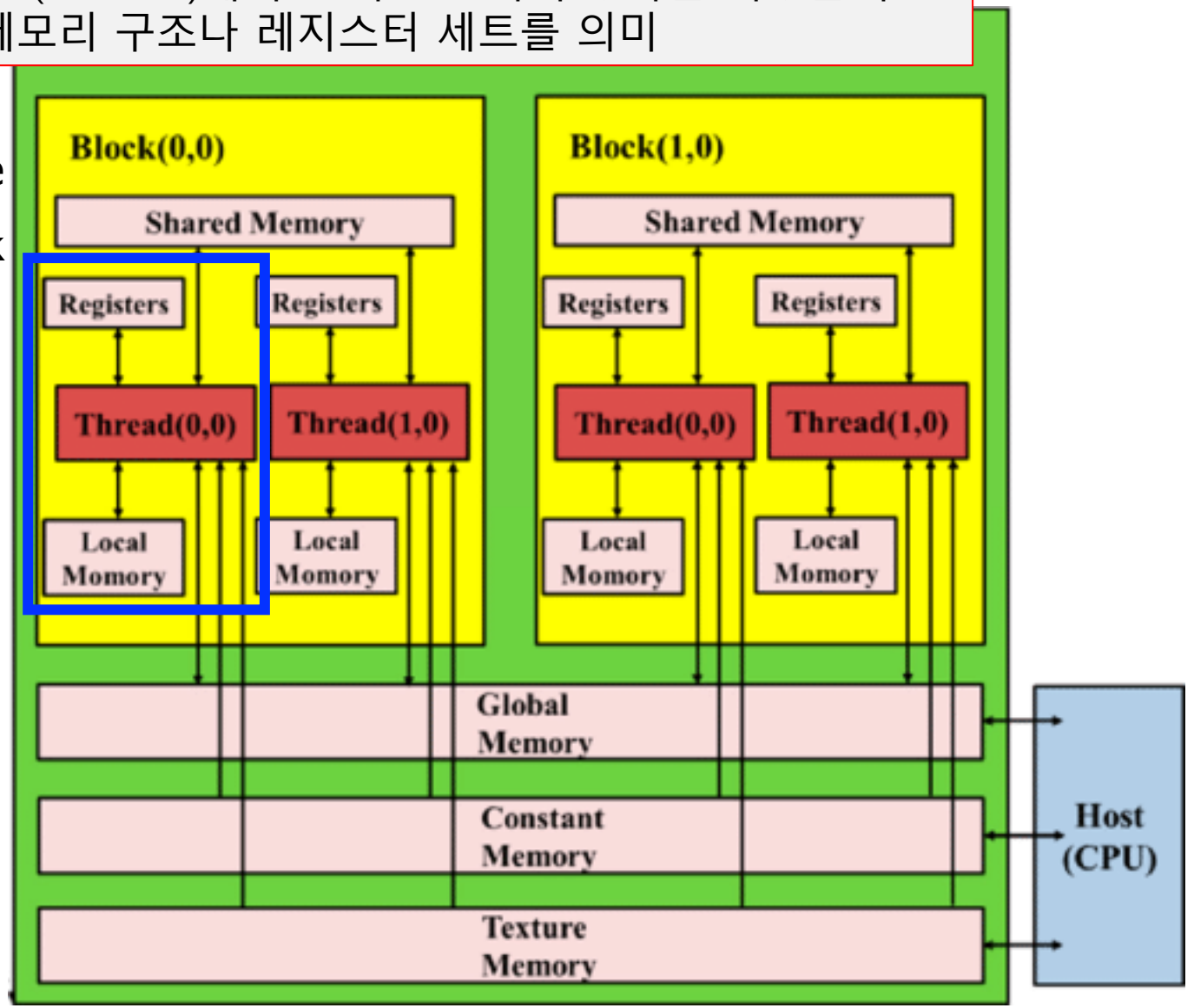
- Per-thread local memory
  - Register
  - Local Memory
- Per Block shared memory
  - Shared memory
- Per Grid global memory
  - Global Memory
  - Constant Memory (read only)
  - Texture Memory (read only)



CUDA Memory Hierarchy – Per thread memory

- Register (in core memory)
  - The fastest, but smallest memory
    - access speed == almost one GPU cycle
    - e.g., 8k – 64k 32 bit registers per block (SM)
  - local variables declared in a kernel by complier.
  - Automatically set by compilers
    - Up to 255 registers per thread
  - Reside in each SM
  - Partitioned by threads in a block
    - Zero context switching

64*1024(=65536)개의 32비트 크기의 항목을 저장할 수 있는 메모리 구조나 레지스터 세트를 의미



CUDA Memory Hierarchy – Per thread memory

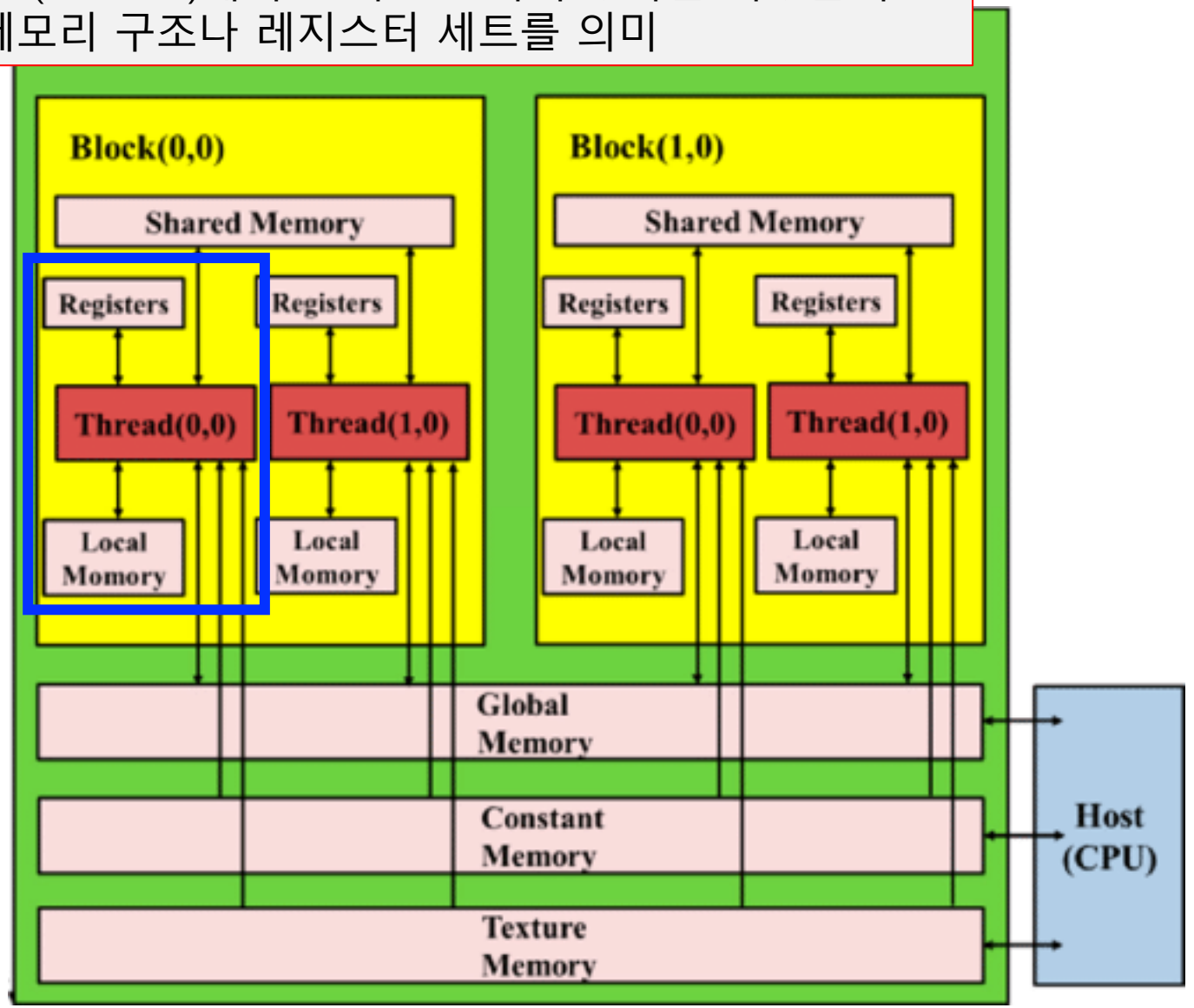
- Register (in core memory)
  - thread-private : 한 thread 만 접근가능
  - 같은 block 안의 다른 thread 도 접근 불가
  - Spilling: Register 가 부족하면, 초과분은 local memory 로 할당함 → 성능저하
- 하나의 SM에서 총 register 갯수는 제한됨.

  (e.g., 64K registers per SM)
  - block / thread 당 사용하는 register 수가 많으면, 동시에 올릴 수 있는 thread/block 수가 줄어듬.
  - more register usage : less thread usage
  - 하드웨어 성능에 맞게 적절히 사용해야함.
- 작성한 Kernel 함수가 느리면,

  1) register 갯수 확인, 2) 불필요한 변수/구조체 확인,

  3) block, thread 크기 조절하여 spill 감소유도

64*1024(=65536)개의 32비트 크기의 항목을 저장할 수 있는 메모리 구조나 레지스터 세트를 의미



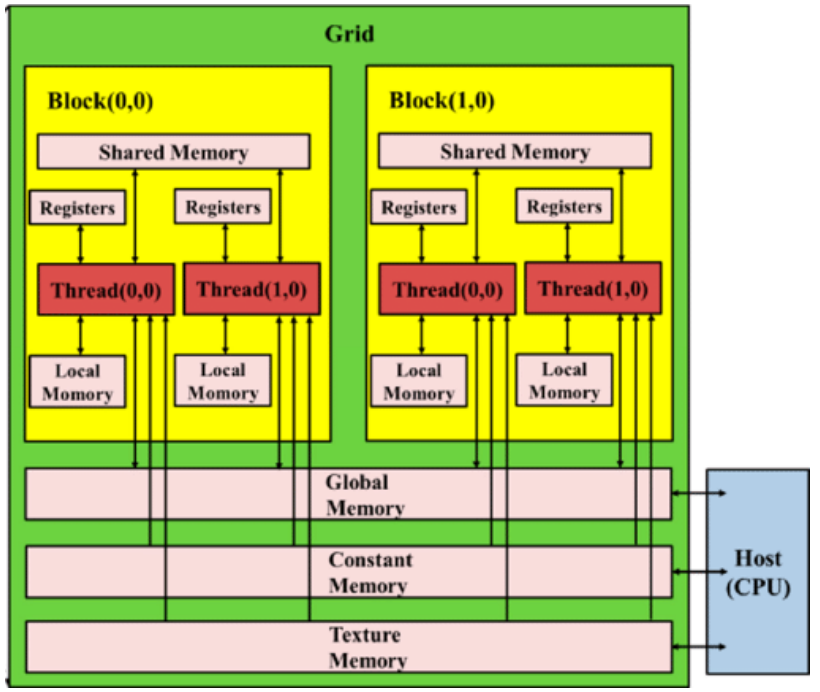
CUDA Memory Hierarchy – Per thread memory

- Local memory (off-chip memory)
  - **Outside of a SM, reside in DRAM**

(이름은 local 이지만 실제로는 global memory device DRAM에 저장 : global memory 수준으로 느림, 다만 L1/L2캐시 작용으로 인해 나아질 수 있음.)

  - 한 thread 안에서만 보이는 변수들(지역 변수, 임시 변수)을 위해 쓰이는 메모리 공간
  - Slow, but Large Space
    - 400~600 GPU cycles
  - Hold local variables
    - Local arrays
    - Large local arrays consume too much register space
    - Any variable does not fit within the register limit
    - 인덱스가 runtime에 결정되는 경우

```
float a[1000];
int idx = threadIdx.x;
float v = a[idx];
```



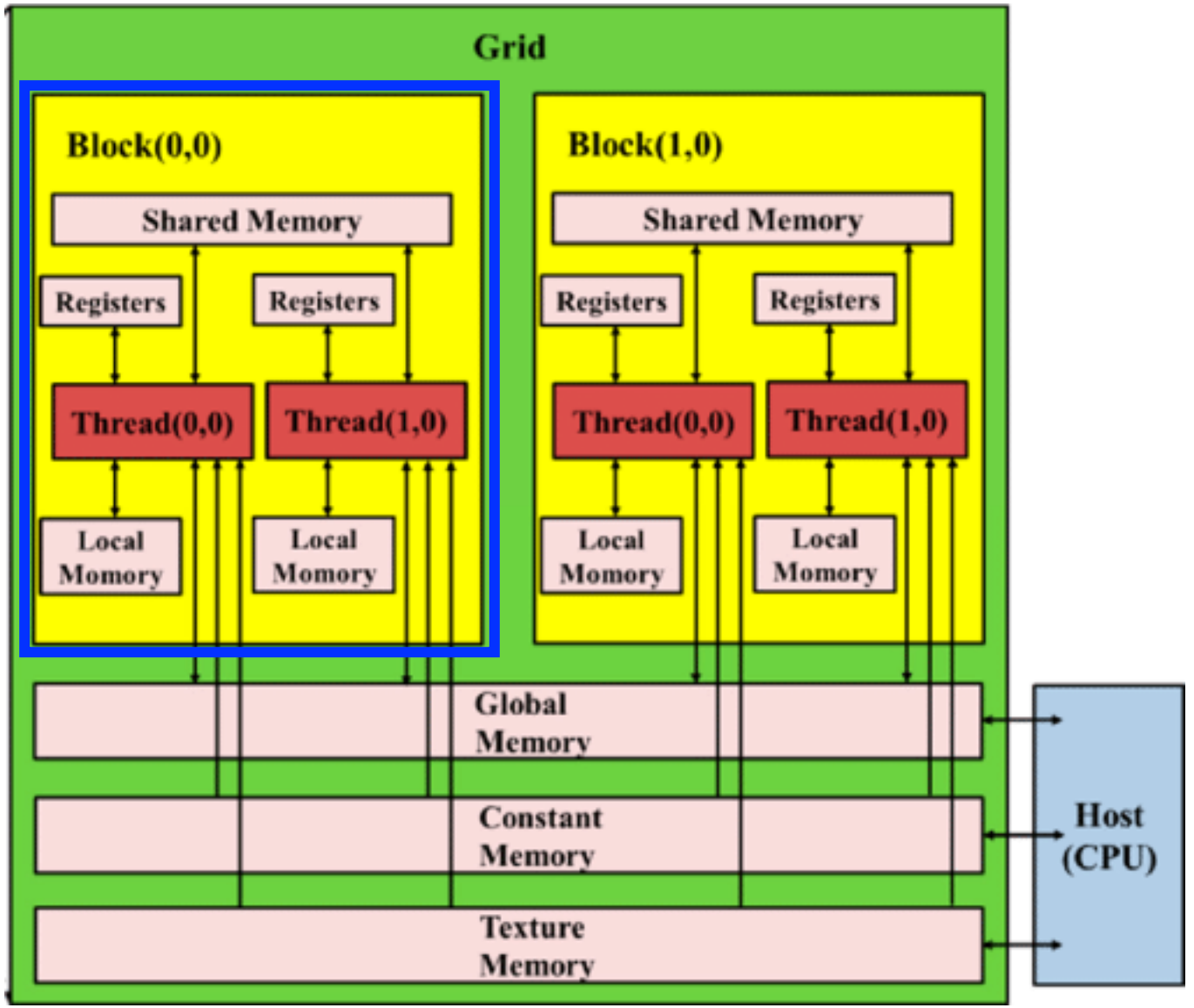
CUDA Memory Hierarchy – Per thread memory

Aspect	Register	Local memory
Scope	thread 전용	thread 전용
Physical location	On-chip	Off-chip
Speed	Almost GPU 1 cycle	Similar with global mem.
Capacity	Very limited (fixed total per SM)	Much larger in principle (bounded by global memory size, but slower)
Allocation policy	Assigned automatically by the compiler to variables (단일 변수 위주)	Used automatically when registers spill or for large/array variables
Addressing	Typically fixed; not suited for arbitrary indexing	General memory addressing; supports arbitrary indexing like arrays
How to use	Overuse → register pressure, spills, lower occupancy	Overuse → many global-like accesses, potential large performance loss

- thread 안에서 너무 큰 local array / structure 사용 자제
- arbitrary indexing (동적인덱싱) 줄이고, 가능하면 작은 요소 줄여 Unrolling / 상수화
- 정말 필요한 thread-private 가 아니면 shared memory로 옮기기 (block 단위 공유)

CUDA Memory Hierarchy – Per block memory

- Shared memory (__shared__)
  - Fast, but small on-chip memory
    - 1~4 GPU cycles
    - 16~96KB per block/SM
  - Shared by all threads in a block
    - All thread in a block can access the same data
    - Inter—thread communication (thread 간 데이터 통신)



CUDA Memory Hierarchy – Per block memory

- Shared memory (__shared__)
  - Fast, but small on-chip memory
    - 물리적으로 on-chip에 있는 SRAM
    - 1~4 GPU cycles
    - 16~96KB per block/SM
  - Shared by all threads in a block
    - All thread in a block can access the same data
    - Inter—thread communication (thread 간 데이터 통신)

- Static allocation

```
__global__ void kernel(void)
{
    __shared__ int sharedMemory[512];
}
```

__shared__ 키워드로 선언
: block 내의 모든 thread 가 buf[] 를 볼 수 있음.

block내 동기화 필수
: read/write 맞추어야함.

```
__global__ void kernel(...) {
    __shared__ float buf[256];  // shared memory (block 내에서 공유)
    int tid = threadIdx.x;

    // global → shared 로 로드
    buf[tid] = some_global_array[tid];
    __syncthreads();            // 모든 thread 로드 끝날 때까지 기다리기

    // 이후 buf[] 를 여러 번 사용
}
```

CUDA Memory Hierarchy – Per block memory

- Shared memory (__shared__)
  - Fast, but small on-chip memory
    - 물리적으로 on-chip에 있는 SRAM
    - 1~4 GPU cycles
    - 16~96KB per block/SM
  - Shared by all threads in a block
    - All thread in a block can access the same data
    - Inter—thread communication (thread 간 데이터 통신)

- Dynamic allocation

```
__global__ void kernel(...) {
    extern __shared__ float sdata[]; //dynamic shared memory

    int tid = threadIdx.x;
    sdata[tid] = tid;
    __syncthreads();
    ...
}
```

크기는 나중에 코드 런칭될 때 정해짐.

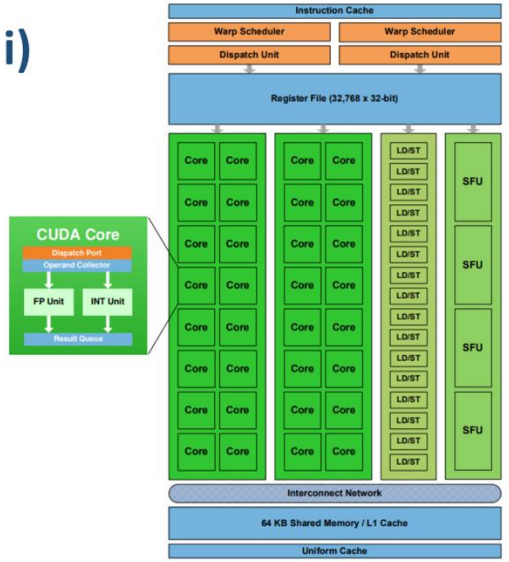
```
int main(void) {
    int blockDim = 256;
    size_t shmBytes = blockDim * sizeof(float);

    kernel<<<gridDim, blockDim, shmBytes>>>(...);
}
```

세번째 인자가 한 block 당 shared memory 바이트수, 예시에서는 각 block 이 float 256개 짜리 sdata[] 를 하나씩 가짐.

CUDA Memory Hierarchy – Per block memory

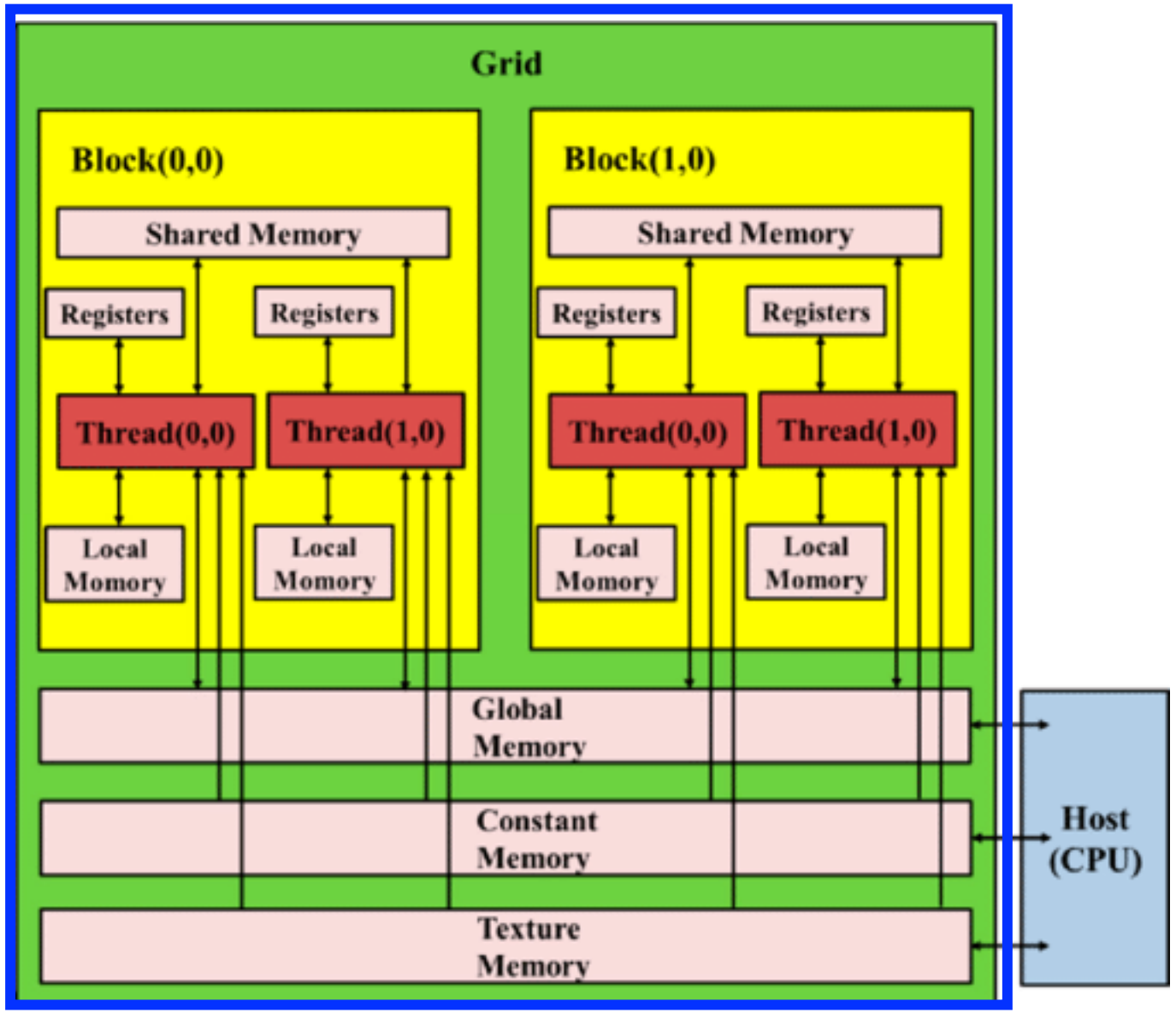
- Shared memory (__shared__)
  - 잦은 Global access 메모리 감소 시켜, 속도 향상
  - 같은 데이터를 여러 thread 가 반복적으로 사용할 때, global 이 아니라 shared 를 사용함.
    - e.g., convolution operation
  - Thread 간 빠른 데이터 교환 기능
  - 큰 데이터를 타일 단위로 나누어서 메모리 접근 패턴 균일화 단순화

- Block 단위로 크게 제한. 많이 쓰면 SM에서 사용할 수 있는 block 수의 감소. i)



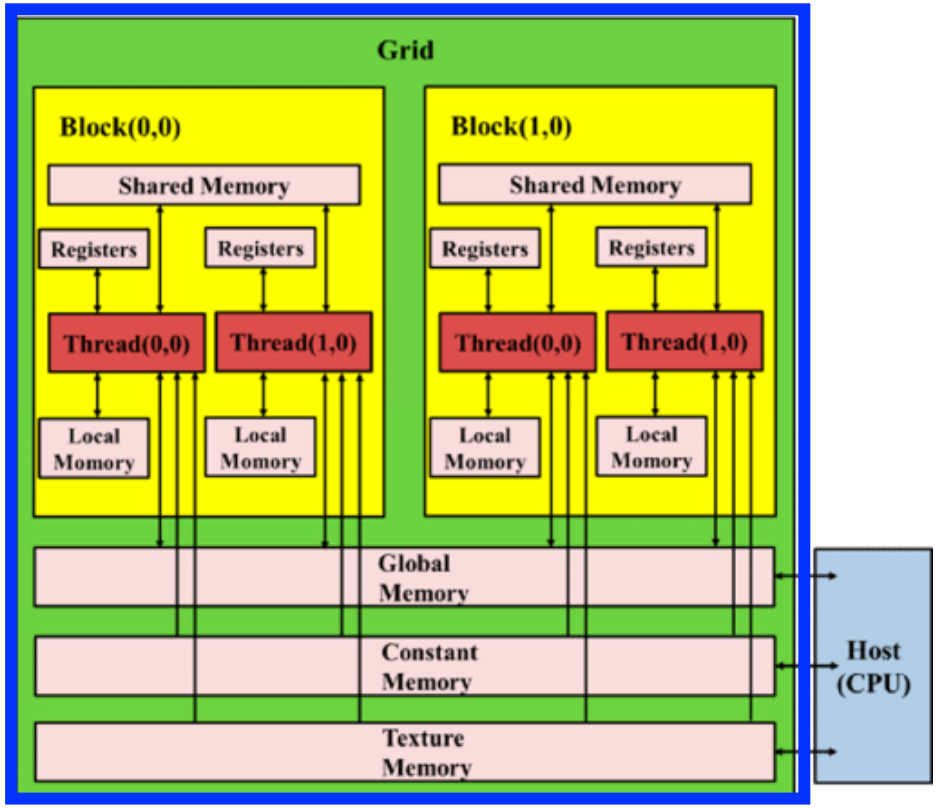
CUDA Memory Hierarchy – Per Grid memory

- Global memory
  - The slowest, but largest off-chip memory
    - 400~800 GPU cycles
    - 8, 16, … GB
  - Shared by all thread in a grid
  - Host also can access



CUDA Memory Hierarchy – Per Grid memory

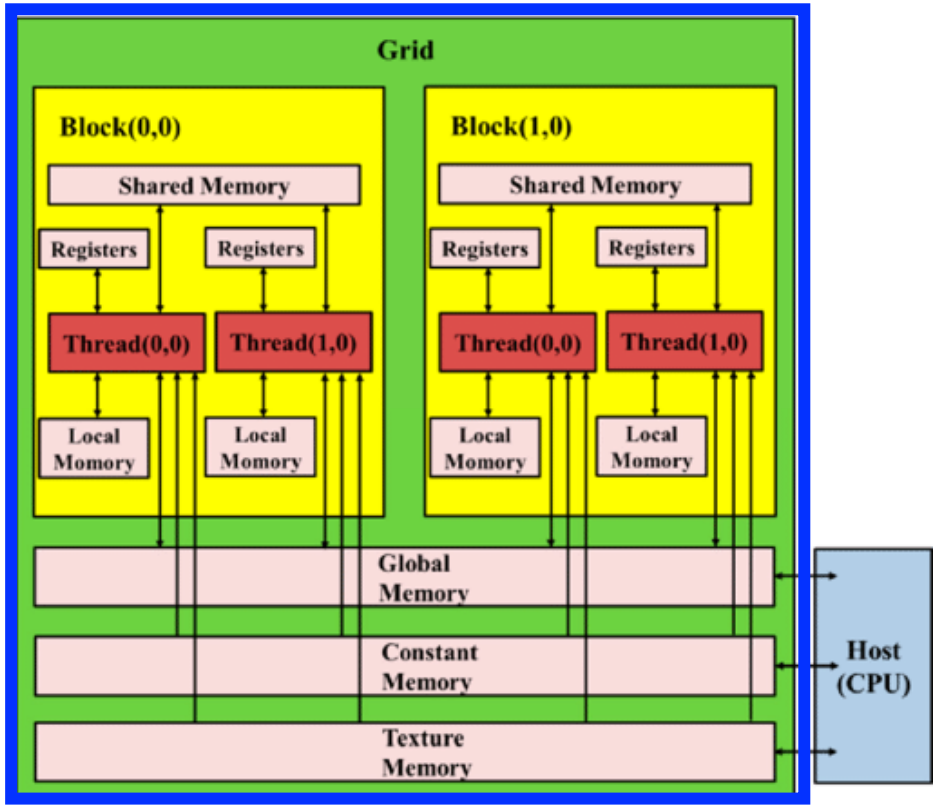
- Constant memory (__constant__)
  - Read-only memory
    - Declared with global scope, and initialized by the host (host 에서만 쓰기 가능)
  - Small but Fast
    - Reside in DRAM, but has dedicated on-chip cache (64KB per SM)
    - 한 warp 안의 모든 thread 가 같은 주소를 읽을 때, 한번의 fetch 로 32개의 thread 로 모두 전달가능 (register 수준으로 빠름)
    - warp내의 thread 들이 각자 다른 주소를 read 할때, 메모리 접근이 serialization 되어서 느려짐.
      → 즉, 모두가 같은 값을 읽을 때 가장 빠름.
  - 사용예시
    - 커널 전체에서 공유하는 상수 파라미터, 작은 lookup table
    - 예: 필터 계수, 변환 행렬, configuration 값들




```
__constant__ int constMemory[512];

int main(void)
{
    int table[512] = { 0 };
    cudaMemcpyToSymbol("constMemory", table, sizeof(int) * 512);
    ...
    kernel <<<gridDim, blockDim>>> ();
}
```

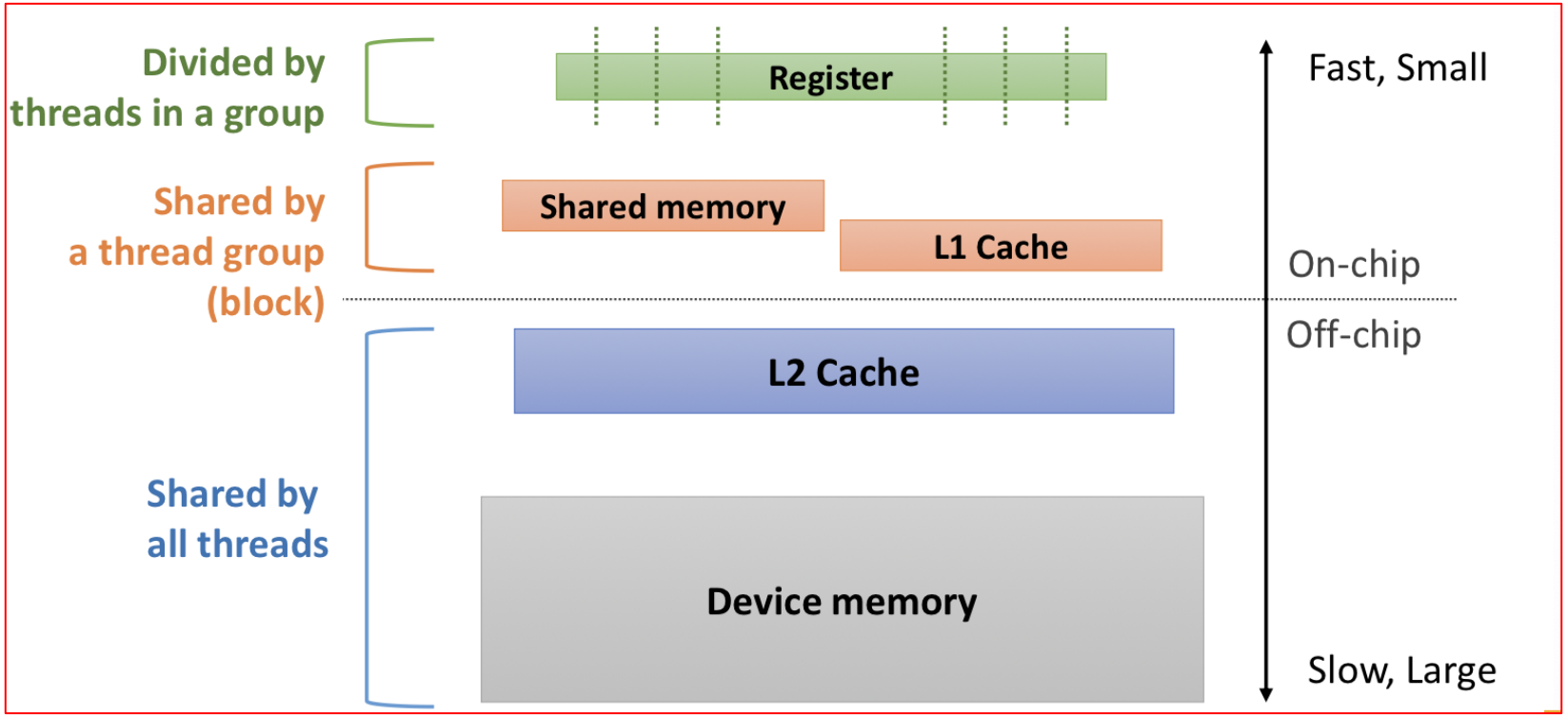
CUDA Memory Hierarchy – Per Grid memory

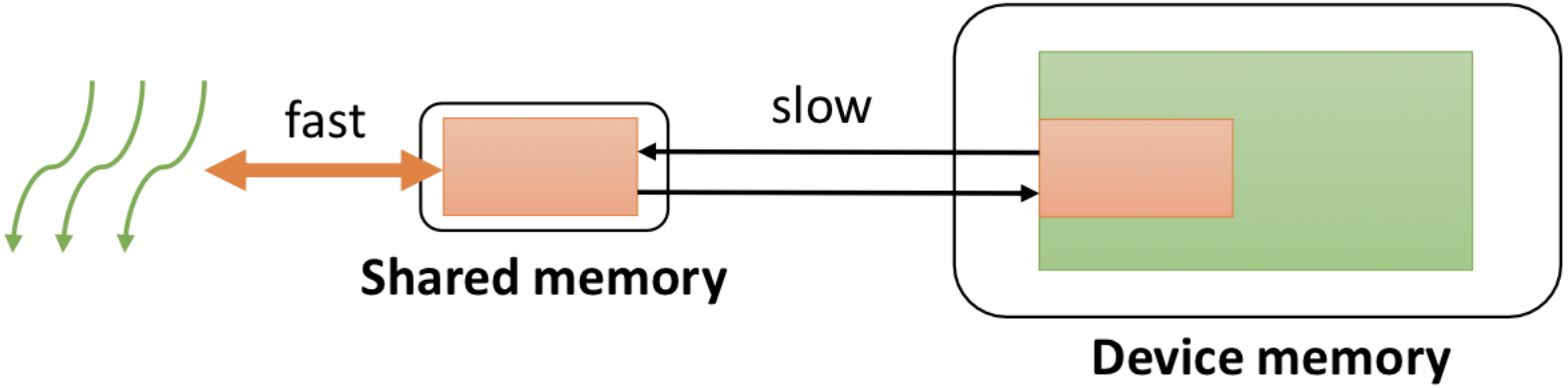
- Texture memory
  - Read-only memory
  - Small but Fast
    - Reside in DRAM, but has dedicated on-chip texture cache
  - Optimized for 2D/3D spatial locality 에 맞게 설계됨.
  - Support hardware filtering
    - 그래픽스 형식의 보간 함수 (linear interpolation) 기능 등
    - thread 들이 2D / 3D 주변 값을 자주 읽는 경우 (공간적 근접성)
    - Global memory 내에 존재 (공간적 근접성이 있는 경우 성능 좋음)
    - 최신 CUDA에서 L1/L2 캐시가 좋아져서 최근에는 자주 사용되지 않음.)



CUDA Memory Hierarchy – Lifetime

- Thread
  - Register
  - Local memory
- Block
  - Share Memory
- Host Allocation
  - Global
  - Constant
  - Texture





Summary

- CUDA Thread
- CUDA Memory Hierarchy
- Brief Introduction on CUDA Memory and parallelism